

Functional Programming

Contents

1	Functional programming	3
1.1	Pure functions	3
1.2	Functional languages	3
1.2.1	Example: computing $n!$	3
1.3	Types	4
1.3.1	Type enforcement	5
1.3.2	Type checking	5
1.3.3	Type notation in Haskell	5
1.4	Currying	6
1.5	λ -calculus	6
1.5.1	λ -expressions	7
1.5.2	Associativity	7
1.5.3	Free vs. Bound variables	7
1.5.4	Transformation rules	7
1.5.5	λ -expressions in Haskell	9
1.5.6	Translating between named functions and λ -expressions	10
1.6	Lists	10
1.7	Pattern matching	10
1.7.1	Pattern matching on lists	11
1.7.2	Matchable patterns	12
1.8	List comprehensions	13
1.8.1	Multiple generators	13
1.8.2	Guards	14
1.9	Function guards	14
1.10	Recursion	14
1.11	Equivalence of recursion and iteration	15
1.12	Stacks	16
1.13	Types of recursive function	17
1.13.1	Linear recursion	17
1.13.2	Multiple recursion	17
1.13.3	Direct recursion	17
1.13.4	Mutual/indirect recursion	17
1.13.5	Tail recursion	17
1.14	Lazy evaluation	18
1.14.1	Function evaluation strategies	18
1.15	Haskell types	21

1.15.1	Adding new types	21
1.15.2	data	22
1.15.3	Type theory	24
1.16	Polymorphism	25
1.16.1	Constrained polymorphism	25
1.16.2	Class and instance	26
1.16.3	Basic type classes	26
1.16.4	Deriving type class	26
1.16.5	Defining classes	26
1.16.6	Creating instances	26
1.16.7	Updating interface and types	27
1.16.8	Default implementation	27
1.17	Higher-order functions	27
1.17.1	map	28
1.17.2	filter	28
1.18	Function composition	28
1.18.1	Function composition in Haskell	29
1.18.2	Point-free style	29
1.19	Functors	30
1.19.1	fmap	30
1.19.2	Functor laws	30
1.19.3	Instance declaration for Functors	30
1.19.4	Generic code using Functors	31
1.20	Applicatives	31
1.20.1	Applicative class	32
1.20.2	Applicative style	32
1.20.3	Applicative functors	32
1.21	Monads	34
1.21.1	Maybe monad	35
1.21.2	Example: safe division	35
1.21.3	Monad laws	36
1.21.4	List monad	36
1.22	IO monad	36
1.23	Folds	37
1.23.1	foldl	37
1.23.2	foldr	38
1.23.3	Purpose of folds	39
1.23.4	foldr or foldl	39
1.24	Traversables	39
1.24.1	sequenceA	39
1.24.2	traverse	40
1.24.3	Relationship between sequenceA and traverse	40

1 Functional programming

Functional programming is a programming paradigm where computation is done by applying functions to arguments (inputs). Programs specify what computations should take place (not necessarily how, as in imperative programming), with a focus on combining and chaining functions. It is similar in style to algebra.

1.1 Pure functions

Look at the Python code below.

```
state = 0

def f(n):
    global state
    state += 1
    return n + state

print(f(1)) # 2
print(f(1)) # 3
```

Here, making the same function call `f(1)` twice produces two distinct outputs. This is because `f` has a **side effect**:

- A side effect is any change a function makes to state or the outside world beyond returning its result.

A **pure function** is a function that takes all its inputs as arguments and produces all its outputs as results. It does not have any side effects.

1.2 Functional languages

A **functional programming language** is a programming language that supports and encourages the functional programming paradigm, where function application and composition are the basic building blocks.

A language is **purely functional** if it forbids variable assignment and side effects, ensuring all functions are pure functions. This makes reasoning about the code simpler for both humans and the compiler.

Haskell is an example of a purely functional language.

1.2.1 Example: computing $n!$

Using an imperative style, $n!$ may be calculated using a for loop:

```
def calculate_factorial(n):
    factorial = 1
    for i in range(1, n+1):
        factorial = factorial * i
    return factorial
```

However, this uses variable assignment (`factorial = factorial * i`), which is forbidden in a

purely functional language. Instead, $n!$ can be calculated recursively, noting the piecewise representation of the factorial function:

$$F_n = \begin{cases} 1 & n = 1 \\ n \times F_{n-1} & \text{otherwise} \end{cases}$$

Pseudocode for this is given below:

```
def factorial(n):
    if n == 1:
        return 1
    else:
        return n * factorial(n-1)
```

This can be implemented in Haskell using the code given below.

Note that comments in Haskell use `--`.

```
[1]: factorial :: Int -> Int -- declare factorial as a function that takes an int
    ↪as input, and outputs an int
factorial 1 = 1 -- declare that factorial of 1 is 1
factorial n = n * factorial(n-1) -- declare that factorial of n is n *
    ↪factorial of n-1

factorial 5 -- call the function with input value 5. Note that arguments do
    ↪not need to be placed in brackets
```

120

1.3 Types

A **type** is a collection of related values.

A type can be described by specifying:

- the set of values it contains, and
- the operations that can be performed on those values

For example, the `Bool` type contains the values `True` and `False`, and supports logical operations such as `&&` (and), `||` (or) and `not`.

In Haskell, every expression (including functions) has a type. For example, `factorial :: Int -> Int` declares that the `factorial` function takes an `Int` as input and returns an `Int` as output.

Types are needed to:

- tell us how to interpret a variable
- define which operations are valid on a value
- help the compiler or interpreter find the correct implementation of an operator
- check if an operation on data is valid and well-defined (ensure code fragments are type-safe)
- document code clearly

1.3.1 Type enforcement

Programs can be classified according to how strongly they enforce types:

- Strongly typed languages enforce types strictly
 - incompatible types cannot be mixed without explicit conversion
 - Haskell is a strongly typed language
- Weakly typed languages do not enforce types strictly
 - the language may automatically convert between types (type coercion)
 - Python is a weakly typed language

1.3.2 Type checking

Translators must check for type correctness (verifying that operations are used with the correct types).

Programming languages can be classified according to when they check for type correctness:

- Statically typed language: type safety is checked at translation time
 - invalid types result in translation error
 - Haskell is a statically typed language
- Dynamically typed language: type safety is checked at run time
 - invalid types are only detected when they are used
 - Python is a dynamically typed language

There are three main ways a translator can determine the type of an expression:

- Explicit annotation: the programmer annotates all variables with type information (e.g. C/Java)
- Type inference: the translator deduces types automatically from how variables are used (e.g. Haskell)
- Duck typing: the translator/runtime tries the operation, and if it succeeds, the type is valid (e.g. Python)

An example of type inference in Haskell is shown below.

```
[2]: x :: Int
     x = 6
     y :: Int
     y = 7
     z = x + y
     -- since x and y both have type Int
     -- and z is the addition of these,
     -- the translator deduces that z must also be an Int
     :t z
```

```
z :: Int
```

1.3.3 Type notation in Haskell

In Haskell, two colons `::` are used to declare type:

- `e :: T` means that `e` is of type `T`

It is possible to check the type of an expression using `:t expr` or `:type expr`.

```
[3]: x :: Int
     x = 1
     :t x

     -- the not logical operator takes a Bool,
     -- and returns a Bool
     :type not
```

```
x :: Int
```

```
not :: Bool -> Bool
```

1.4 Currying

Currying is a way of writing functions so that they take one argument at a time and return a new function for the next argument.

- this means that multi-argument functions are actually a sequence of single-argument functions

Almost all functions in Haskell are written in the curried form.

```
-- "uncurried"
add :: (Int, Int) -> Int -- input is a tuple containing two Ints
add (x, y) = x + y

-- "curried"
add_curried :: Int -> Int -> Int -- input is an Int, returns a function that takes
                                -- another Int, which then returns an Int
add_curried x y = x + y
```

The `->` symbol associates to the right, removing the need for brackets:

- `Int -> Int -> Int -> Int` is equivalent to `Int -> (Int -> (Int -> Int))`

Function application associates to the left:

- `mult x y z` is equivalent to `((mult x) y) z`

1.5 λ -calculus

λ -calculus is a formal system for describing computation using only:

- function definition (abstraction)
- function application

In λ -calculus:

- everything is a function
- there are no variables with changing state
- computation consists of applying functions to arguments

It forms the theoretical foundation of functional programming, and is the simplest known Turing-complete system (i.e., it can compute anything that any computer can in theory).

1.5.1 λ -expressions

λ -expressions can be defined inductively: - A variable v is a λ -expression - variables are represented by lower-case letters - If M is a λ -expression, then $(\lambda v.M)$ is a λ -expression - this defines a function: - v is the parameter - M is the function body - λ is the start of the function definition - e.g. $\lambda x.x + 1$ - If M and N are λ -expressions, then (MN) is a λ -expression - this means that the function M is applied to the argument N

Note that functions take exactly one argument; currying is used to model multi-argument functions.

Example of valid λ -expressions include:

- x : a variable
- $(\lambda x.x)$: the identity function (returns its input)
- $((\lambda x.x) e)$: the identity function applied to an expression e
- $(\lambda x.(\lambda y.(xy)))$: a (nested/curried) function that takes x and returns another function that takes y and applies x to y
- $((\lambda x.(\lambda y.(xy))) a) b$: a nested function applied to expressions a and b . This reduces to (ab) , i.e., the function a applied to the argument b

1.5.2 Associativity

Application is left-associative:

- MNP means apply M to N , and then apply that result to P , i.e., $((MN)P)$

Abstraction (function definition) is right-associative:

- $\lambda x.\lambda y.M$ means $\lambda x.(\lambda y.M)$

This eliminates the need to use brackets.

Application takes precedence over abstraction:

- $\lambda x.\lambda y.xy$ is equivalent to $\lambda x.(\lambda y.(xy))$, rather than $\lambda x.(\lambda y.x)y$

1.5.3 Free vs. Bound variables

A variable is **bound** if it is a parameter of an enclosing function (λ -abstraction). - for instance, x in $\lambda x.x$ is bound - a variable binds to the closest enclosing λ -abstraction that declares the same variable name

A variable is **free** if it is not bound - for instance, x in $\lambda y.x$ is free (and y is bound)

1.5.4 Transformation rules

β -reduction β -reduction is used to evaluate λ -expressions by substituting the argument of a λ -abstraction into its body:

$$((\lambda x.M[x]) N) \rightarrow M[x := N]$$

When an argument is applied (substituted), the outer λ -abstraction is eliminated.

Here:

- $\lambda x.M[x]$ is a λ -abstraction (function) with parameter x and body $M[x]$ (like a function $f(x) = M[x]$)
- N is the argument being applied to the function
- $((\lambda x.M[x]) N)$ is applying the function to argument N
- $M[x := N]$ is obtained by replacing every occurrence of the parameter x in $M[x]$ with the actual argument N

For example:

- $(\lambda x.x) a = a$
 - replacing x with a in the body gives a
- $(\lambda x.y) a = y$
 - there is no occurrence of x in the body, so the result is y

A more complex, nested example is:

$$\begin{aligned} ((\lambda x.(\lambda y.(xy))) a) b &= \\ ((\lambda y.(ay)) b) &= \\ ab \end{aligned}$$

A λ -expression is in β -normal form if no β -reduction is possible (the expression cannot be reduced further).

α -conversion α -conversion is used to rename bound variables in a λ -abstraction.

$$(\lambda x.M[x]) \rightarrow (\lambda y.M[x := y])$$

It replaces a bound variable with a fresh variable name. - The structure and meaning of the expression remain unchanged.

This helps to avoid name conflicts and prevent variable capture during β -reduction:

- a name conflict happens when two different variables use the same name, even though they refer to different things
 - e.g., in the expression $\lambda x.(\lambda x.x)$, the outer x and inner x are different variables, but share the same name
- variable capture is when a free variable becomes bound accidentally during substitution (β -reduction)
 - e.g., take the expression $(\lambda x.(\lambda y.xy)) y$
 - * substituting $x := y$ would produce $\lambda y.yy$
 - * the free y (the argument) that was substituted in is now bound by λy
 - * the meaning of the expression has changed

α -conversion renames the bound variable to prevent this from happening:

$$(\lambda x.(\lambda y.xy)) y \rightarrow (\lambda x.(\lambda z.xz)) y$$

Now, β -reduction produces:

$\lambda z.yz$

y now stays free; the semantics of the expression is preserved.

1.5.5 λ -expressions in Haskell

As well as creating named functions as was done earlier, nameless functions can be constructed using λ -expressions.

A λ -expression has the form:

```
\x -> expression
```

Below is code for a nameless function that takes a number x and returns $x + x$

```
\x -> x + x
```

Use example 1 Unnamed functions can be used to formalise the idea of functions defined using currying:

```
add x y = x + y
-- is equivalent to
add = \x -> (\y -> x + y)
```

The second form emphasises the idea that `add` is a function of one variable that returns a function (which takes the second variable).

Use example 2 Unnamed functions can also be useful when returning a function as a result:

```
const :: a -> b -> a
const x _ = x
```

The `const` function takes two inputs, and always returns the first, ignoring the second. This can be made much simpler using λ -notation:

```
const :: a -> b -> a
const x = \_ -> x
```

This makes it clear that `const` takes its first argument and returns a function `\_ -> x`. This function can then take a second argument, which it ignores, and always returns the first argument `x`.

Use example 3 Another use is for when a function is used only once.

```
odds :: Int -> [Int]
odds n = map f [0..n-1]
  where
    f x = x*2 + 1
```

This function generate the first n positive odd numbers.

Using a λ -function, can simplify this function, removing the entire `where` clause:

```
odds :: Int -> [Int]
odds n = map (\x -> x*2 + 1) [0..n-1]
```

1.5.6 Translating between named functions and λ -expressions

Any multi-argument function can be rewritten using λ -expressions by moving arguments one-by-one to the right-hand side and wrapping each in a λ function.

```
f a b c = ...  
f = \a b c -> ...  
f = \a -> (\b -> (\c -> ...))
```

Both forms are equivalent; the choice of which to use is stylistic.

1.6 Lists

In Haskell, every non-empty list is built using the `cons` (construct) operator `:`. This operator takes an element and a list, and prepends the element to the list (adds it to the front of the list).

Although lists are written just using square brackets, this is just syntactic sugar for repeated use of the `:` operator.

```
[1,2,3,4]  
-- is syntactic sugar for  
1 : (2 : (3 : (4 : [])))
```

Elements can be accessed using the `!!` operator to index the array:

```
[4]: x = [1,2,3,4]  
x !! 2 -- indexing is 0-based
```

3

Lists in Haskell are representations of (singly) linked lists:

- operations on lists such as indexing, or computing the length must therefore traverse the list
 - therefore operations like `reverse`, `length`, `++` (concatenation) and `!!` are linear in the length of the list
- getting the `head` and `tail` is constant time, as is `:` itself.

1.7 Pattern matching

Pattern matching in Haskell is a way of deconstructing values and binding their parts to names based on their shape.

Instead of writing explicit conditionals (like `if`), functions are defined by matching different patterns of input.

In a function, there can be multiple equations, each handling a different pattern:

```
f 0 = "zero"  
f 1 = "one"  
f _ = "something else"
```

Here, `0` and `1` are specific patterns, and `_` is a wildcard, which matches anything.

Pattern matches are tried from top to bottom, and the first match is used.

Pattern matches should cover all possible cases; otherwise, a runtime error may occur.

1.7.1 Pattern matching on lists

Pattern matching in lists can be used to recognise specific shapes of lists and extract parts from them.

The patterns must be used in the equations that define the function, not in the type of the function. Pattern matches in the equations don't change the type of the function - they just say how it should act on particular expressions.

In the example below, `['a', _, _]` is a pattern for a list with exactly three elements. It matches a list where the first element is 'a' (the other two elements can be anything).

```
[5]: startsWithA :: [Char] -> Bool
      startsWithA ['a', _, _] = True
      startsWithA _ = False

      startsWithA ['a','b','b']
      startsWithA ['b','b','b']
```

True

False

Note that **Strings** are just a list of **Chars**, so pattern matching on lists works on strings: - "abc" is exactly the same as `['a','b','c']` - double quotes "" are used for **Strings**, and single quotes '' for **Chars**

```
[6]: startsWithA "abb" -- "abb" is the same as ['a','b','b']
```

True

This function only works for lists of length 3; for a list that starts with 'a' but doesn't have length 3, it will return **False**:

```
[7]: startsWithA "a"
      startsWithA "aaaaa"
```

False

False

In order to match lists of any length, the `:` operator can be used with the wildcard `_`:

```
[8]: startsWithA' :: [Char] -> Bool
      startsWithA' ('a': _) = True
      startsWithA' _ = False
```

`('a': _)` matches any list of at least length 1 whose first entry is 'a'.

```
[9]: startsWithA' "aaaaa"
```

True

This method also works to match multiple entries:

```
[10]: startsWithAB :: [Char] -> Bool
      startsWithAB ('a':'b':_) = True
      startsWithAB _ = False

      startsWithAB "abcdef"
```

True

Binding variables As well as just checking for specific literal values, pattern matching can be used to deconstruct data and bind parts of it to variables.

```
sumTwo :: Num a => [a] -> a
sumTwo (x:y:_) = x + y
```

In this function: - (x:y:_) is a pattern that matches any list of length at least 2 - the first element of the list is bound to x - the second element is bound to y - the rest of the list is ignored (using _) - the sum of x and y is returned

Note that variable names in a single pattern must be unique, though the wildcard can be used multiple times (to ignore the part it matches).

1.7.2 Matchable patterns

Pattern matching works by mirroring the structure of the data.

As a general rule, patterns are constructed to look like the arguments that would be passed to the function.

It can be used to match: 1. Literal patterns - matching specific values directly

```
(&&) :: Bool -> Bool -> Bool
True && True = True
False && _ = False
-- matches Boolean literals `True` and `False`
```

2. Constructor patterns

- matching data built using constructors (e.g., lists)

```
head :: [a] -> a
head (x:_) = x
-- matches any non-empty list
-- binds first element to `x`, ignores rest
```

It is not possible to match arbitrary expressions in patterns; only literals and data constructors are allowed:

```
last :: [a] -> a
last (xs ++ [x]) = x -- not allowed
-- the ++ operator concatenates two lists
```

Here, xs ++ [x] is an expression, not a constructor. Pattern matching only works with the structure of the data, not general computations.

1.8 List comprehensions

List comprehensions in Haskell are a concise way to build lists by describing their elements (similar to set notation in maths).

A list comprehension has the syntax:

```
[ expression | generator, guard ]
```

- **expression** determines what gets added to the output list
- a **generator** has the form `x <- list` and draws values from a list, binding each value to a variable
- **guards** are optional, and can be used to filter the generated elements

```
[11]: doubles = [x * 2 | x <- [1..5]] -- no condition
doubles
```

```
[2,4,6,8,10]
```

`x <- [1..5]` draws values of `x` from the list `[1..5]`, and `x * 2` doubles them.

A guard could be used to only include even values of `x`:

```
[12]: evenDoubles = [x * 2 | x <- [1..5], even x] -- filter to only include even x
evenDoubles
```

```
[4,8]
```

1.8.1 Multiple generators

A list comprehension can contain multiple generators, separated by commas.

Each generator introduces a new variable, and generators are evaluated from left to right. Later generators behave like inner loops and change faster.

```
[13]: tuples = [(x,y) | x <- [1,2], y <- [3,4]]
tuples
```

```
[(1,3),(1,4),(2,3),(2,4)]
```

This works like a `for` loop:

```
for x in [1,2]:
    for y in [3,4]:
        ...
```

It is also possible for a later generator to use variables from earlier ones:

```
[14]: tuples' = [(x,y) | x <- [1..3], y <- [x..3]]
tuples'
```

```
[(1,1),(1,2),(1,3),(2,2),(2,3),(3,3)]
```

This is equivalent to the `for` loop:

```
for x in range(1,4):
    for y in range(x,4):
        ...
```

1.8.2 Guards

Guards are Boolean conditions used to filter results: - a guard can be any function that returns a Bool - guards can only use variables that have already been introduced (to their left)

```
[15]: evenOdd = [(x,y) | x <- [1..5], even x, y <- [1..5], odd y]
evenOdd
```

```
[(2,1),(2,3),(2,5),(4,1),(4,3),(4,5)]
```

This combination of filters creates a list of the tuples (x,y), where x is even and y is odd, and both are between 1 and 5.

1.9 Function guards

In functions, the | symbol is used to introduce guards. These are conditional branches that are evaluated top-to-bottom until one condition evaluates to **True**.

Take the below Python function:

```
def sign(x):
    if x > 0:
        return 1
    elif x < 0:
        return -1
    else:
        return 0
```

This can be implemented in Haskell using guards:

```
sign x
| x > 0 = 1
| x < 0 = -1
| otherwise = 0
```

Note that **otherwise** does not have any special meaning; it is conventionally used to represent the fallback case (it is simply a predefined name from the **Prelude**, defined as **otherwise = True**).

1.10 Recursion

Recursion means to define something in terms of itself.

When defining a recursive function in Haskell, the following steps can be followed:

1. define the type
2. enumerate the cases
3. define the simple (base) cases
4. define how other cases reduce to simpler ones
5. (optional) generalise and simplify

For example, suppose we want a function `drop` which drops the first `n` elements from a list. Then, we can follow each of these steps to produce a recursive function:

1. define the type

```
drop :: Int -> [a] -> [a]
```

2. enumerate the cases

```
-- consider all combinations of inputs
-- integer: 0 or non-zero
-- list: empty [] or non-empty (x:xs)
drop 0 [] =
drop 0 (x:xs) =
drop n [] =
drop n (x:xs) =
```

3. define the simple (base) cases

```
-- if n = 0, no elements are dropped, so return the original list
-- if the list is empty, there is nothing to drop, so return []
drop 0 [] = []
drop 0 (x:xs) = x:xs
drop n [] = []
drop n (x:xs) =
```

4. define the reduction of other cases to simpler ones

```
-- if n is non-zero and the list is non-empty
-- remove the first element
-- apply drop (n-1) to the rest of the list
drop 0 [] = []
drop 0 (x:xs) = x:xs
drop n [] = []
drop n (x:xs) = drop (n-1) xs
```

5. generalise and simplify

```
-- combine cases to make the definition more concise
-- if n = 0, return the list unchanged
-- if the list is empty, always return []
-- otherwise, recurse on the tail
drop 0 xs = xs
drop _ [] = []
drop n (x:xs) = drop (n-1) xs
```

1.11 Equivalence of recursion and iteration

Recursion and iteration are computationally equivalent: any algorithm expressed using one can be transformed into the other.

- the more convenient approach depends on the algorithm and the programming language

To convert from recursion to iteration:

- simple recursion can often be rewritten using loops and variables
- more complex recursion may require an explicit stack to simulate the call stack

To convert from iteration to recursion:

- loop variables become function parameters
- loop is expressed as recursive calls
- use a tail recursive function (where recursive call is the final operation in the function)

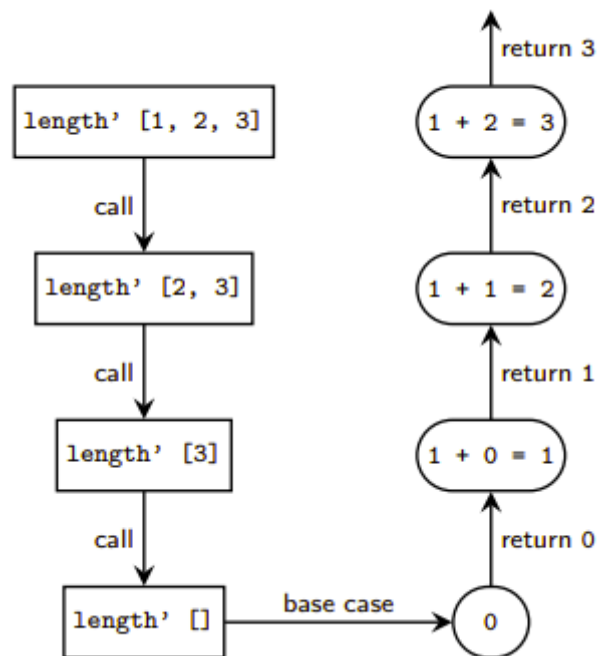
1.12 Stacks

A stack is used to manage function calls (including nested and recursive calls). - a stack is a linear data structure following the Last-In/First-Out (LIFO) principle, where the last element added is the first one removed

Take the function below:

```
length' :: [a] -> Int
length' [] = 0
length' (x:xs) = 1 + length' xs
```

The diagram below illustrates how recursive calls to `length'` are pushed onto the call stack and how results are returned as the stack unwinds when `length' [1,2,3]` is called.



Each entry on the stack uses memory:

- too many entries causes errors (such as stack overflow)
- the size of the stack depends on the language
 - it is typically small in imperative languages and larger in functional ones

1.13 Types of recursive function

Recursive functions can be classified in different ways, which helps when designing efficient implementations.

1.13.1 Linear recursion

A function is linearly recursive if each call makes at most one recursive call.

```
length' [] = 0
length' (_:xs) = 1 + length' xs
```

The function calls itself repeatedly until it hits the base case.

1.13.2 Multiple recursion

A function is multiply recursive if each call makes more than one recursive call.

```
fib 0 = 0
fib 1 = 1
fib n = fib (n - 1) + fib (n - 2)
```

This often leads to exponential time complexity due to repeated work.

1.13.3 Direct recursion

A function is directly recursive if it calls itself.

```
product' [] = 1
product' (x:xs) = x * product' xs
```

Direct recursion can be either linear or multiple, depending on how many recursive calls are made.

1.13.4 Mutual/indirect recursion

Functions are mutually recursive if they call each other.

```
even' :: Integral a => a -> Bool
even' 0 = True
even' n = odd' (n - 1)
```

```
odd' :: Integral a => a -> Bool
odd' 0 = False
odd' n = even' (n - 1)
```

1.13.5 Tail recursion

A function is tail recursive if the recursive call is the final operation, meaning its result is returned directly with no further computation.

```
drop 0 xs = xs
drop _ [] = []
drop n (x:xs) = drop (n-1) xs
```

Tail-recursive functions are important because:

- they can be optimised to run in constant stack space
- they correspond closely to loops in imperative languages

Tail recursion and iteration are equivalent:

- tail recursion can be translated into loops
- any loop can be rewritten as a tail-recursive function

Consider the following iterative factorial in Python:

```
def factorial(n):
    res = 1
    for i in range(n, 1, -1):
        res *= i
    return res
```

This can't be written directly in Haskell because variables are immutable (so reassignment like `res *= i` is not allowed). However, it can be turned into a tail recursive function by using a helper recursive function where all the variables in the loop (`res` and `i`) become a parameters:

```
factorial n = loop n 1
  where loop n res
    | n < 0 = undefined
    | n > 1 = loop (n - 1) (res * n)
    | otherwise = res
```

1.14 Lazy evaluation

Haskell is evaluated lazily:

- in **lazy evaluation**, expressions are not evaluated when they are bound to variables; they are evaluated only when their value is required

1.14.1 Function evaluation strategies

Haskell's basic method of computation is application of functions to arguments. There are two main ways to do this.

Look at the example function and call below:

```
inc :: Int -> Int
inc n = n + 1  -- increment input

inc (2 * 3)  -- function call with argument 2*3
```

Call-by-value The arguments are evaluated first, and then the function is applied.

```
inc (2 * 3)
= inc 6
= 6 + 1
= 7
```

Call-by-name Arguments are substituted into the function body without being evaluated, and evaluation is performed only when required.

```
inc (2 * 3)
= (2 * 3) + 1
= 6 + 1
= 7
```

As long as evaluation terminates (and there are no side effects), the evaluation order does not affect the result.

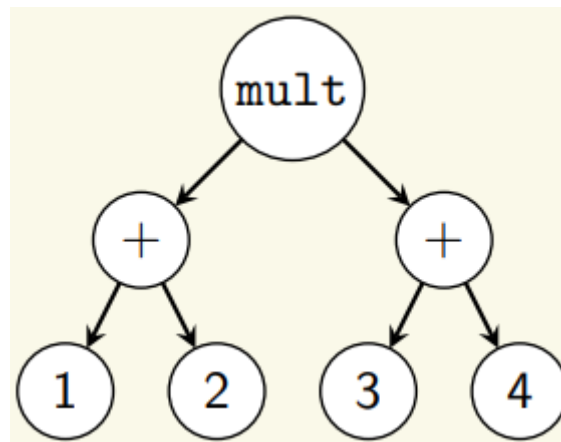
Computation as a graph A function call and its arguments can be represented as a graph.

Take the function and call below:

```
mult :: (Int, Int) -> Int
mult (x, y) = x*y
```

```
mult (1+2, 3+4)
```

This can be represented by the graph below:

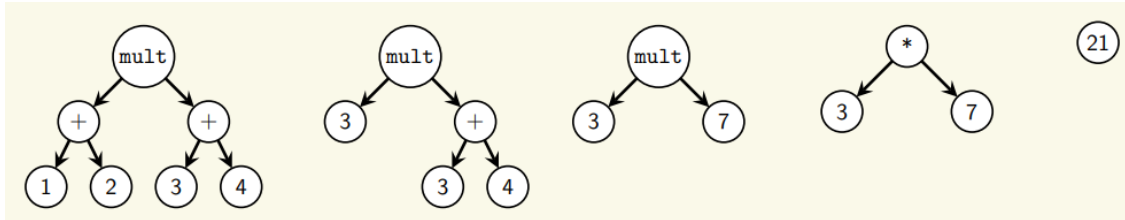


Nodes in the graph are either terminals or compounds:

- a terminal node represents an expression that cannot be reduced any further (it is already a final value)
 - 1, 2, 3 and 4 are terminals
- a compound node represents an expression that is made up of smaller expressions and can still be evaluated (reduced). These are also called reducible expressions (redexes)
 - + and mult represent compound nodes

Innermost evaluation In innermost evaluation:

- the graph is evaluated bottom-up
- innermost (deepest) redexes are evaluated first (i.e. those whose arguments are already evaluated values)
- evaluation is typically left-to-right along leaves, though this is implementation-dependent

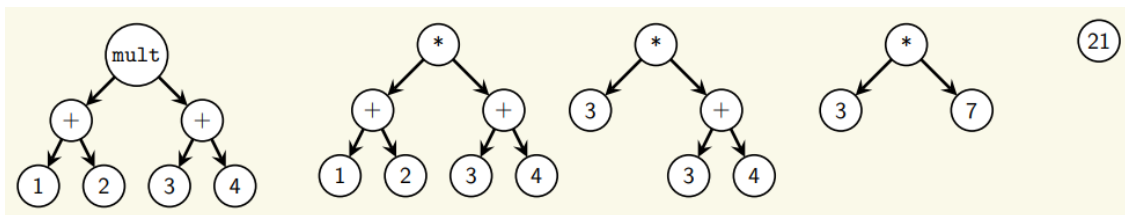


Innermost evaluation corresponds to the call-by-value method for evaluating functions:

- arguments are evaluated before the function is applied
- this is used in most imperative languages

Outermost evaluation In outermost evaluation:

- the graph is evaluated top-down
- outermost redexes are evaluated first
- inner expressions are only evaluated if required by the outer expression
- evaluation is typically left-to-right along children, though this is implementation-dependent



Outermost evaluation corresponds to the call-by-name method for evaluating functions:

- each argument may be evaluated more than once
- Haskell uses a modified version of this (call-by-need) which avoids recomputation by caching results

Termination For expressions that terminate, both innermost and outermost evaluation produce the same result. However, for some expressions involving non-terminating computations, innermost evaluation may fail to terminate, whereas outermost evaluation can still produce a result if the non-terminating part is not required.

Take the example below:

```
inf :: Integer
inf = 1 + inf
fst :: (a, b) -> a
fst (x, _) = x

fst (0, inf)
```

With innermost evaluation, the arguments to `fst` are evaluated first. This requires evaluating `inf`, which does not terminate.

With outermost evaluation, the `fst` function is applied first. This shows that the second argument is not needed, so `inf` is never evaluated, and the result is 0.

1.15 Haskell types

There are many commonly used built-in types in Haskell:

- **Int**: Fixed-precision integers (bounded; typically 64-bit)
- **Integer**: Arbitrary-precision integers (can grow as large as memory allows)
- **Float**: Single-precision floating-point numbers
- **Double**: Double-precision floating-point numbers
- **Char**: A single Unicode character
- **Bool**: Booleans, with values **True** and **False**, conjunction (**&&**), disjunction (**||**), and negation **not**
- **Lists**: Any sequence of elements of the same type, e.g. `[1, 4, 5]` or `['b', 'd']`
 - Strings are lists of characters, so `"bd"` has type `[Char]` and is equivalent to `['b','d']`
- **Tuples**: Fixed-size collections that may contain elements of different types., e.g. `(1, "hello")`
 - Functions such as **curry** and **uncurry** convert between tupled and curried function forms

1.15.1 Adding new types

There are many reasons to define a new data type:

- Hide complexity
- Build higher-level abstractions
- Improve type safety

Haskell provides three ways to create new data types:

- **type**
- **data**
- **newtype**

type The **type** keyword is used to define a type synonym (or type alias). This gives an existing type a new name, usually to make code easier to read and understand.

For example:

```
type Name = String
type Age  = Int

type Person = (Name, Age)

person1 :: Person
```

This is equivalent to:

```
person1 :: (String, Int)
```

However, the first version is clearer because it communicates the meaning of each component of the tuple.

Type synonyms can be parameterised by one or more type variables.

```
type Pair a = (a, a)
```

Here, `Pair a` is a type synonym for for a tuple containing two values of the same type. When a concrete type is supplied, it is substituted into the definition:

- `Pair Int` becomes `(Int, Int)`
- `Pair Char` becomes `(Char, Char)`

Limitations of `type` include:

- It does not create a new type; it only creates a new name for an existing type
 - `type Age = Int`
 - `Age` and `Int` are treated identically by the compiler
- It cannot enforce class constraints
 - `type Number a = a` does not ensure that `a` is numeric
- It cannot define recursive types
 - `type Tree = (Int, [Tree])` is invalid

1.15.2 data

A completely new type can be formed using the `data` declaration.

```
data Bool = False | True
```

This creates a type `Bool` whose only possible values are `False` and `True`:

- these values are called constructors
- constructors are separated using the `|` symbol
- both type names and constructor names must begin with an uppercase letter

Constructors represent the different ways a value of a type can be formed.

Constructor parameters Each constructor can carry zero or more values (parameters).

```
data Shape
  = Circle Float
  | Rectangle Float Float
```

This defines a type `Shape` with two constructors:

- `Circle` takes one `Float` (the radius)
- `Rectangle` takes two `Floats` (width and height)

Each constructor is a different way of constructing a `Shape`, and the values it carries represent the data stored in that shape.

Pattern matching on constructors Values of a `data` type can be deconstructed using pattern matching on constructors.

```
area :: Shape -> Float
area (Circle r) = pi * r^2
area (Rectangle x y) = x * y
```

Pattern matching works by:

- checking which constructor was used to create the value
- extracting the data stored inside that constructor

For example, `(Circle r)` matches a `Circle` value and binds the radius to `r`.

Polymorphic data types `data` declarations can be polymorphic by using type variables. This allows the same structure to work with many different types.

```
data Maybe a = Nothing | Just a
```

A value of type `Maybe a` is either:

- `Nothing`, representing the absence of a value
- `Just x`, representing a value `x` of type `a`

Here, `a` is a type variable, meaning `Maybe` can be used with any type.

A common use of `Maybe` is representing computations that may fail to produce a result.

For example, take the function:

```
safehead :: [a] -> Maybe a
safehead [] = Nothing
safehead (x:_) = Just x
```

The function attempts to get the first element of a list:

- if the list is empty, there is no first element, so the result is `Nothing`
- if the list is non-empty, the first element is returned wrapped in `Just`
- it is polymorphic, meaning it works for lists of any type `a`

Recursive types `data` declarations can refer to themselves. Such definitions are called recursive types.

Recursive types allow values to contain other values of the same type, making it possible to represent structures of arbitrary size such as natural numbers, lists, and trees.

Take the example below:

```
data Nat = Zero | Succ Nat
```

The `Nat` data type defines the natural numbers. It has two constructors:

- `Zero`, representing 0
- `Succ Nat`, representing the successor of another natural number

This means that every `Nat` is either:

- `Zero` (0)
- `Succ n`, where `n` is another value of type `Nat`.

Each natural number is therefore built by repeatedly applying `Succ` to `Zero`.

```
Zero    -- 0
Succ Zero    -- 1
Succ (Succ Zero)  -- 2
...
```

Since there is no limit to how many times `Succ` can be applied, this definition represents an infinite set of possible values.

Recursive types allow for very concise definitions of data structures:

```
data List a = Empty | Cons a (List a)

intList = Cons 1 (Cons 2 (Cons 3 Empty))
-- represents [1, 2, 3]
```

A value of type `List a` is either:

- `Empty`, representing an empty list
- `Cons a (List a)`, representing one element of type `a`, followed by another `List a`

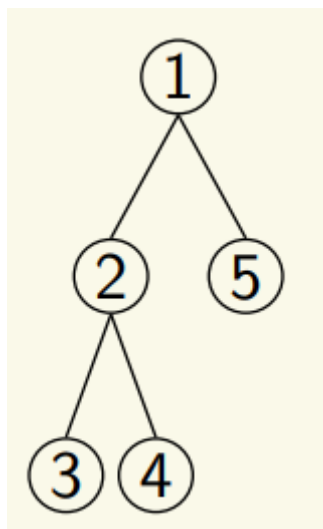
Essentially, a list is either empty, or it consists of a head element and a tail list.

Binary tree A slightly more complex example of the use of recursive types is creating a binary tree.

```
data BTree a
  = Empty
  | Node a (BTree a) (BTree a)
-- a BTree is either:
--   - empty, or
--   - a node containing a value,
--     a left subtree, and a right subtree

btree = Node 1 (Node 2 (Node 3 Empty Empty)
                      (Node 4 Empty Empty))
        (Node 5 Empty Empty)
```

This would form the binary tree below:



1.15.3 Type theory

Haskell's `data` declarations define algebraic data types:

- this is a type where the shape of each element is specified

- the two algebraic operations are ‘sum’ and ‘product’
 - a sum is a choice between alternatives
 - * e.g., `data Foo = A | B`
 - * a value of type `Foo` can either be `A` or `B`
 - a product is a collection of values grouped together
 - * e.g., `data Pair = P Int Double`
 - * a value of type `Pair` is two numbers, an `Int` and `Double` together

Pros and cons of algebraic data types Algebraic data types make it easy to define new operations over an existing set of constructors:

- to add a new operation, simply write a new function that pattern matches on the data type

However, they make it harder to add new kinds of values:

- to add a new kind of value, a new constructor must be added to the data declaration
- any existing functions that pattern match on the type may also need to be updated to handle the new constructor

For example, using the `Shape` type from earlier:

```
data Shape
  = Circle Float
  | Rectangle Float Float
```

- to add a new shape like a triangle, a new constructor would have to be added, and the `area` function would need to be updated to handle the new shape
- adding a new function such as `perimeter :: Shape -> Float` does not require any changes to the `Shape` type itself

1.16 Polymorphism

Polymorphism is the ability for a function or expression to be used with values of different types.

Haskell has parametric polymorphism:

- this is when a single implementation of a function can apply generally and identically to values of any type

1.16.1 Constrained polymorphism

Some polymorphic functions only work for types that satisfy specific type class constraints.

```
(+) :: Num a => a -> a -> a
```

This means that `(+)` works for any type `a`, provided that `a` is an instance of the type class `Num`.

The part `Num a =>` is called a type class constraint:

- functions with constrained types are called overloaded functions, and types containing class constraints are called overloaded types
 - `Num a => a -> a -> a` is an overloaded type
 - `(+)` is an overloaded function

1.16.2 Class and instance

Note that the meanings of class and instance in functional programming differ from their meanings in object-oriented programming. In Haskell:

- A type class is a collection of types that support a common set of overloaded operations called methods, e.g., `Num`
- An instance is a specific type that belongs to a type class and provides implementations of the required methods, e.g., `Int` is an instance of `Num`

1.16.3 Basic type classes

There are a number of common type classes defined in the `Prelude`:

- `Eq`: types whose values can be tested for equality and inequality (`==`, `/=`)
- `Ord`: types whose values can be compared and placed in a total ordering (a subclass of `Eq`)
- `Read`: types whose values can be parsed from strings
- `Show`: types whose values can be converted to strings
 - types must have this class to be able to be output to the terminal
- `Num`: numerical types, that allow addition and multiplication

1.16.4 Deriving type class

A data type can automatically become an instance of certain type classes using the `deriving` keyword.

```
data Shape
  = Circle Float
  | Rectangle Float Float
  deriving (Show)
```

Without a `Show` instance (either derived or manually defined), it would not be possible to convert values of type `Shape` to strings or print them using functions such as `show` or `print`.

1.16.5 Defining classes

An type class (interface) can be defined using the `class` keyword. It declares a set of methods that types must implement.

```
class Foo a where
  isfoo :: a -> Bool
```

- `Foo` is the type class name
- `a` is a type variable
- `where` introduces the class methods
- `isfoo` is a class method

1.16.6 Creating instances

Instances define how specific types implement the methods of a type class.

```
[16]: class Foo a where
      isfoo :: a -> Bool

      instance Foo Int where
        isfoo _ = False

      instance Foo Char where
        isfoo c = c `elem` ['a'..'c']
```

This defines:

- Int as an instance of Foo, where isfoo always returns False
- Char as an instance of Foo, where isfoo returns True for characters 'a' to 'c', and False otherwise

```
[17]: isfoo (5 :: Int)
```

False

```
[18]: isfoo 'b'
```

True

1.16.7 Updating interface and types

In Haskell, it is possible to both:

- add new type class instances for existing types
 - e.g., define a new instance of a type class for Int
- define instances of existing type classes for new data types
 - e.g., make a new type an instance of Num

1.16.8 Default implementation

Type classes can provide default implementations for methods.

For example, the Eq class (representing equality) requires both equality (==) and inequality (/=) to be defined.

Since $a == b \iff \text{not } (a /= b)$, the Eq class can provide default implementations, so that each instance only needs to implement one, from which the other can be defined automatically.

```
class Eq a where
  (==) :: a -> a -> Bool
  x == y = not (x /= y)
  (/=) :: a -> a -> Bool
  x /= y = not (x == y)
```

1.17 Higher-order functions

A higher-order function is a function that does at least one of the following:

- take one or more functions as arguments

- returns a function as its result

Due to currying, every function of more than one argument is higher-order in Haskell.

Many linear recursive functions on lists can be written using higher order library functions. For example:

- `map` applies a function to each element in the list
- `filter` selects elements from a list that satisfy a condition

1.17.1 `map`

`map` takes a function and a list as arguments, and applies the function to every element in the list:

```
map :: (a -> b) -> [a] -> [b]
map _ [] = []
map f xs = [f x | x <- xs]
```

- the first input is a function `(a -> b)`
- the second is a list `[a]`
- when the list is empty, return `[]`
- otherwise, construct the return list by applying the function to each element of the list

1.17.2 `filter`

`filter` takes a predicate (expression that evaluates to `True` or `False`) and a list as inputs and returns the list containing only those elements that evaluate to `True` under the predicate.

```
filter :: (a -> Bool) -> [a] -> [a]
filter _ [] = []
filter p xs = [x | x <- xs, p x]
```

- the first input is a predicate `(a -> Bool)`
- the second is a list `[a]`
- when the list is empty, return `[]`
- otherwise, construct the list containing only those elements for which the predicate evaluates to `True`

1.18 Function composition

Function composition is a way of combining smaller functions into a larger function.

In mathematics, function composition is written as

$$(f \circ g)(x) = f(g(x))$$

This means:

- apply g to x
- apply f to the result

So the function g runs first, even though it appears second.

1.18.1 Function composition in Haskell

Haskell uses the `(.)` operator for function composition.

```
(.) :: (b -> c) -> (a -> b) -> (a -> c)
```

This type means:

- the first function has type `b -> c`
- the second function has type `a -> b`
- the result is a new function of type `a -> c`
 - this is because the output of the second function (`b`) becomes the input to the first function

The `.` operator is defined as:

```
f . g = \x -> f (g x)
```

This means:

- take an argument
- apply `g` to `x`
- apply `f` to the result

So `(f . g) x` is exactly the same as `f (g x)`.

For example, take the function:

```
odd a = not (even a)
```

This can be rewritten using composition:

```
odd = not . even
```

1.18.2 Point-free style

The definition:

```
odd = not . even
```

is an example of point-free style, where the argument (`a`) is omitted.

Point-free style can make code shorter and emphasize how functions are combined rather than how data flows through explicit variables.

The function

```
odd = not . even
```

expands to:

```
odd = \x -> not (even x)
```

which is equivalent to

```
odd x = not (even x)
```

1.19 Functors

A **Functor** is a type class that represents types whose contained values can be transformed.

```
class Functor f where
    fmap :: (a -> b) -> f a -> f b
```

A type constructor **f** is a functor if it provides an implementation of **fmap** and is of type

```
f :: * -> *
```

i.e., it takes exactly one type parameter.

1.19.1 fmap

In the type:

```
fmap :: (a -> b) -> f a -> f b
```

- `(a -> b)` is a function that transforms values
- `f a` is a value of type `a` inside some context `f`
- `f b` is the same context containing transformed values

So **fmap**:

- applies a function to each value inside the structure
- preserves the structure itself

Therefore, **fmap** applies a function to the contents of a structure.

For example, lists are Functors:

```
fmap (+1) [1,2,3]
-- [2,3,4]
```

is equivalent to

```
map (+1) [1,2,3]
```

1.19.2 Functor laws

Every valid **Functor** instance must satisfy two laws.

- `fmap id c == c`
 - mapping the identity function over a structure should return the structure untouched
- `fmap f (fmap g c) == fmap (f . g) c`
 - mapping `g` and then `f` is the same as mapping the composed function `f . g`

These laws guarantee that **fmap** behaves predictably.

1.19.3 Instance declaration for Functors

An instance declaration is used to attach an **fmap** implementation to a type constructor.

```
data List a = Nil | Cons a (List a)
    deriving (Eq, Show)
```

```

instance Functor List where
    fmap _ Nil = Nil
    fmap f (Cons a tail) = Cons (f a) (fmap f tail)
    -- Empty list maps to empty list.
    -- For a non empty list, match the constructor
    -- which matches the first element, and the
    -- rest of the list.
    -- Apply the transformation to the first element,
    -- and then to the rest of the list recursively.

data BinaryTree a
    = Leaf a
    | Node a (BinaryTree a) (BinaryTree a)
    deriving (Eq, Show)

instance Functor BinaryTree where
    fmap f (Leaf a) = Leaf (f a)
    fmap f (Node a l r) = Node (f a) (fmap f l) (fmap f r)
    -- To transform a leaf node, just apply the
    -- transformation to that leaf node's value.
    -- To transform a non-leaf node, apply the transformation
    -- to that node's value, then recursively apply the
    -- transformation to the node's left and right subtrees

```

1.19.4 Generic code using Functors

Since each Functor has the `fmap` method with known behaviour, generic code can be used to apply functions to any Functor:

```

add1 :: (Functor c, Num a) => c a -> c a
add1 = fmap (+1)

```

This `add1` function works for any Functor that contains numeric values.

1.20 Applicatives

Functors abstract the idea of mapping a function over each element of a structure (e.g. a list).

It is often useful to generalise this idea so that functions with multiple arguments can also be applied inside a context.

This would essentially involve extending `fmap` into a family of functions:

```

fmap0 :: a -> f a
fmap1 :: (a -> b) -> f a -> f b
fmap2 :: (a -> b -> c) -> f a -> f b -> f c
fmap3 :: (a -> b -> c -> d) -> f a -> f b -> f c -> f d
...

```

- `fmap0` takes a value and places it into the context `f`
- `fmap1` is the usual `fmap`

- `fmap2` combines two contexts using a binary (two-input) function
- `fmap3` combines three contexts using a ternary (three-input) function

1.20.1 Applicative class

This generalisation is captured by the `Applicative` type class.

Using the idea of currying, a version of `fmap` for functions of an arbitrary number of arguments can be constructed in terms of two basic functions:

```
pure :: a -> f a
(<*>) :: f (a -> b) -> f a -> f b
```

- `pure` embeds a value into a context `f`
- `(<*>)` applies a function inside a context to a value also inside a context

1.20.2 Applicative style

Using currying, multi-argument functions can be applied inside a context:

```
pure g <*> x1 <*> x2 <*> ... <*> xn
```

where:

- `g :: a -> b -> c -> ... -> r`
- `x1 :: f a`
- `x2 :: f b`
- ...
- result is `f r`

An expression such as this is said to be in applicative style.

This expression is applied as follows:

- lift `g` into context using `pure`
- apply it to `x1`
- the result is a function still inside the context `f`, but with one fewer argument
- apply the result to `x2`
- and so on

1.20.3 Applicative functors

The subclass of functors that support `pure` and `<*>` are called applicatives:

```
class Functor f => Applicative f where
  pure :: a -> f a
  (<*>) :: f (a -> b) -> f a -> f b
```

They must satisfy the applicative laws:

- `pure id <*> x = x`
- `pure (g x) = pure g <*> pure x`
- `x <*> pure y = pure (\g -> g y) <*> x`
- `x <*> (y <*> z) = (pure (.) <*> x <*> y) <*> z`

These laws guarantee that any well-typed expression using `pure` and `<*>` can be written in applicative style.

List applicative The list type is an applicative:

```
instance Applicative [] where
  pure x = [x]
  gs <*> xs = [g x | g <- gs, x <-xs]
```

This means that:

- `pure x` wraps a value into a single-element list
- `<*>` takes a list of functions (`gs`) and a list of values (`xs`), and applies every function to every value, collecting all results in a new list

For example:

```
pure (+) <*> [1,2,3] <*> [44,55]
-- [45,56,46,57,47,58]
```

Maybe as an applicative Maybe can be made into an applicative:

```
instance Applicative Maybe where
  -- pure :: a -> Maybe a
  pure = Just

  -- (<*>) :: Maybe (a -> b) -> Maybe a -> Maybe b
  Nothing <*> _ = Nothing
  (Just g) <*> mx = fmap g mx
```

This works as follows:

- `pure` lifts a value into a successful computation
- `Nothing <*> _`: if the function is missing, the computation fails
- `(Just g) <*> mx`: apply function `g` if it exists otherwise propagate failure

IO applicative The IO (input/output) type can be made into an applicative:

```
instance Applicative IO where
  -- pure :: a -> IO a
  pure = return

  -- (<*>) :: IO (a -> b) -> IO a -> IO b
  mg <*> mx = do {g <- mg; x <- mx; return (g x)}
```

- `pure` lifts a pure value into the IO context
- `<*>` combines two IO actions:
 - `mg :: IO (a -> b)` is an IO action that, when executed, yields a function `a -> b`
 - `mx :: IO a` is an IO action that, when executed, yields a value of type `a`

`<*>` is defined using:

```
mg <*> mx = do
  g <- mg
```

```
x <- mx
return (g x)
```

This means:

- run the IO action `mx` to obtain a value `x`
- apply the function to the value: `g x`
- wrap the result back into IO using `return`

A function that reads a given number of characters from the keyboard can be defined as follows:

```
getChars :: Int -> IO String
getChars 0 = return []
getChars n = pure (:) <*> getChar <*> getChars (n-1)
```

Effectful programming Applicative functors provide a general way to work with effectful computations.

An effectful computation is one that does more than simply return a value. For example, it may:

- fail to produce a result (Maybe)
- produce multiple possible results ([])
- perform input/output (IO)

The same applicative syntax works for all Applicatives, while the specific Applicative instance determines how the effects are combined.

1.21 Monads

A Monad is a type class for sequencing computations in a context (effects), where the result of one computation can influence the next. It is a subclass of `Applicative`:

```
class Applicative m => Monad m where
  (>>=) :: m a -> (a -> m b) -> m b
  (>>)  :: m a -> m b -> m b
  return :: a -> m a
```

In this definition:

- `>>=` is called the bind operator
 - sequences two computations
 - extracts the value from the first context and feeds it into the next computation
 - the second function returns a new monadic value
- `>>` is the bind operator without passing
 - sequences two computations
 - ignores the result of the first computation
- `return` wraps a value in a context, producing a monadic value
 - equivalent to `pure`
 - it does not end function execution like in imperative languages

1.21.1 Maybe monad

The `Maybe` monad represents computations which can fail by not returning a value.

```
data Maybe a = Just a | Nothing

instance Monad Maybe where
  -- (>>=) :: Maybe a -> (a -> Maybe b) -> Maybe b
  Nothing >>= f = Nothing
  Just x >>= f = f x

  -- return :: a -> Maybe a
  return x = Just x
```

This definition means that:

- if the first computation is `Nothing`, stop immediately and return `Nothing`
- if the first computation is `Just x`, extract `x` and apply the function `f` to it
- the result of `f x` (which is another `Maybe`) becomes the result of the whole expression

1.21.2 Example: safe division

When dividing two numbers, there is a possibility of dividing by zero, in which case there will be no result. In this case, it is appropriate to use `Maybe`, signalling the potential for the computation to fail.

```
data Expr = Val Int | Div Expr Expr

safediv :: Int -> Int -> Maybe Int
safediv _ 0 = Nothing
safediv n p = Just (n `div` p)

safeeval :: Expr -> Maybe Int
safeeval (Val n) = Just n
safeeval (Div x y) = do n <- safeeval x
                        p <- safeeval y
                        safediv n p
```

Here:

- `Expr` is a type with two constructors:
 - `Val` takes one `Int`
 - `Div` takes two expressions
- `safediv x y` evaluates to:
 - `Nothing`, if `y = 0`
 - `Just (x \`div\` y)` otherwise
- `safeeval` evaluates an expression inside the `Maybe` monad:
 - if the expression is `Val n`, it returns `Just n`
 - if the expression is `Div x y`, it uses `do` notation to:
 - * first evaluate `x` and call the result `n`
 - * then evaluate `y` and call the result `p`

- * if either fails, the whole computation returns `Nothing`
- * if both succeed, `safediv n p` is applied

1.21.3 Monad laws

Monads must satisfy the following laws:

- Left identity: `return a >>= f = f a`
 - Wrapping a value in a context and binding it to a function is the same as applying the function to the extracted value
- Right identity: `m >>= return = m`
 - Taking a monadic value and binding it to `return` leaves the monadic value unchanged
- Associativity:
 - The order of grouping chained monadic operations does not affect the result.
$$(m \gg= (\lambda x \rightarrow g \ x)) \gg= (\lambda y \rightarrow h \ y)$$

$$\Leftrightarrow$$

$$m \gg= (\lambda x \rightarrow g \ x \gg= (\lambda y \rightarrow h \ y))$$

1.21.4 List monad

The list type is another monad:

```
instance Monad [] where
  -- (>>=) :: [a] -> (a -> [b]) -> [b]
  xs >>= f = [y | x <- xs, y <- f x]
```

`xs >>= f` applies `f` to each element of the list, where `f` produces a list for each element, and then concatenates all results.

1.22 IO monad

The `IO` type is another monad. This allows us to perform sequences of actions with input/output strings of characters. For example:

```
stringToNum :: String -> IO Int
stringToNum s = return (read s)
-- (read s) converts s to an Int
-- return (read s) converts the result to an IO Int

inc :: Int -> IO Int
inc x = return (x + 1)
-- inc takes an int, and increments it
-- returns it as an IO Int

main :: IO ()
main = getLine >>= stringToNum >>= inc >>= print
```

Main works as follows:

- `getLine` produces an `IO String`
- `>>=` passes the resulting `String` to `stringToNum`
- `stringToNum` produces an `IO Int`

- `>>=` passes the resulting `Int` to `inc`
- `inc` produces an `IO Int`
- `>>=` passes the `Int` to `print`
- `print` produces `IO ()`

The same `main` function can be written equivalently using `do` notation:

```
main :: IO ()
main = do
    input <- getLine
    num <- stringToNum input
    result <- inc num
    print result
```

Each `<-` extracts the value from the `IO` action inside the monad, allowing it to be used in the next step.

1.23 Folds

Folds are higher-order functions that process a data structure element by element and combine its contents into a single result.

- a binary function is applied repeatedly
- an accumulator stores the intermediate result
- the final accumulator value is returned

This is similar to the `reduce` function in Python.

For lists, the two most common folds are:

- `foldl`: left-associative fold
- `foldr`: right-associative fold

1.23.1 foldl

`foldl` processes a list from left to right.

```
foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f z [] = z
foldl f z (x:xs) = foldl f (z `f` x) xs
```

`foldl` takes three arguments:

- a combining function `f`
- an initial accumulator value `z`
- a list `[a]`

The recursive definition works as follows:

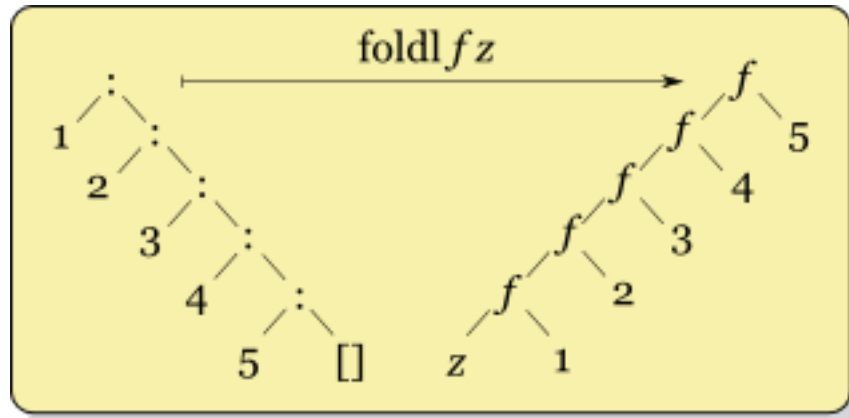
- If the list is empty, return the accumulator `z`
- If the list is non-empty:
 - update the accumulator using `f z x`
 - continue folding over the rest of the list

`foldl` processes the list from left to right:

```
foldl f z [x1, x2, x3]
```

expands to

```
f (f (f z x1) x2) x3
```



1.23.2 foldr

foldr processes a list from right to left.

```
foldr :: (a -> b -> b) -> b -> [a] -> b
```

```
foldr f z [] = z
```

```
foldr f z (x:xs) = f x (foldr f z xs)
```

foldr takes three arguments:

- a combining function `f`
- an initial accumulator value `z`
- a list `[a]`

The recursive definition works as follows:

- If the list is empty, return the accumulator `z`
- If the list is non-empty:
 - recursively fold the tail of the list `xs`
 - apply the function `f` to the head `x` and the result of folding the rest of the list

foldl processes the list from left to right:

```
foldl f z [x1, x2, x3]
```

expands to

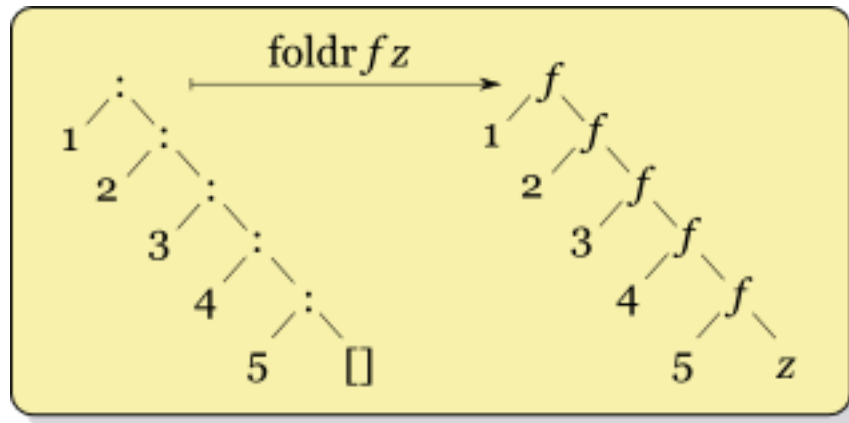
```
f (f (f z x1) x2) x3
```

foldr processes the list from right to left:

```
foldr f z [x1, x2, x3]
```

expands to

```
f x1 (f x2 (f x3 z))
```



1.23.3 Purpose of folds

Folds:

- capture many linear recursive patterns in a clean, reusable way
- enable efficient library implementations, allowing compiler optimisations
- generalise beyond lists: they work for all types that are instances of the `Foldable` type class

1.23.4 foldr or foldl

Generally, `foldr` is the better choice of fold function:

- `foldr` works for infinite lists; `foldl` does not
- `foldr` can terminate early; `foldl` typically cannot
- `foldr` aligns with the structure of lists, making it more natural for many recursive definitions

Note that using `foldr` and `foldl` will lead to different results if `f` is not commutative (i.e., if $f\ x\ y \neq f\ y\ x$).

1.24 Traversable

A `Traversable` type is a data structure that allows effectful computations to be applied to each element in order, collecting the results into a single `Applicative` effect while preserving the structure.

1.24.1 sequenceA

The `sequenceA` method of the `Traversable` class is used to turn a structure of effectful values into a single effect that produces a structure of values, while preserving the order of effects.

```
sequenceA :: Applicative f => t (f a) -> f (t a)
```

Essentially, it:

- takes a container of effects: `t (f a)`
- evaluates each action in the structure, from left to right
- and turns it into one combined effect: `f (t a)`

For example:

```
[19]: sequenceA [Just 1, Just 2, Just 3]
```

```
Just [1,2,3]
```

1.24.2 traverse

`traverse` is used to apply a function that produces effects to each element of a structure, run all those effects, and collect the results back into the same structure.

```
traverse :: Applicative f => (a -> f b) -> t a -> f (t b)
```

It takes two arguments:

- an effectful function `a -> f b`
- a structure of values `t a`

Then it:

- applies the function to every element, producing `t (f b)`
- combines all those `f b` values into one effect: `f (t b)`

The shape `t` stays the same, only values change.

For example:

```
[20]: traverse Just [1,2,3]
```

```
Just [1,2,3]
```

1.24.3 Relationship between `sequenceA` and `traverse`

A Traversable instance can define either `traverse` or `sequenceA`, since each can be defined in terms of the other.

They are equivalent:

```
traverse f = sequenceA . fmap f
```

This means:

- first map the function over the structure, producing `t (f b)`
- then sequence the effects into `f (t b)`

And conversely:

```
sequenceA = traverse id
```

This works because `id` leaves the effectful structure unchanged, so traversal directly sequences the effects.